Analyzing Leakage of Personally Identifiable Information in Language Models

Nils Lukas*§, Ahmed Salem†, Robert Sim†, Shruti Tople†, Lukas Wutschitz† and Santiago Zanella-Béguelin†
*University of Waterloo, †Microsoft Research
nlukas@uwaterloo.ca, {t-salemahmed, rsim, shruti.tople, lukas.wutschitz, santiago}@microsoft.com

2023 IEEE Symposium on Security and Privacy (SP) | 978-1-6654-9336-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/SP46215.2023.10179300

***Abstract*—Language Models (LMs) have been shown to leak information about training data through sentence-level membership inference and reconstruction attacks. Understanding the risk of LMs leaking Personally Identifiable Information (PII) has received less attention, which can be attributed to the false assumption that dataset curation techniques such as scrubbing are sufficient to prevent PII leakage. Scrubbing techniques reduce but do not prevent the risk of PII leakage: in practice scrubbing is imperfect and must balance the trade-off between minimizing disclosure and preserving the utility of the dataset. On the other hand, it is unclear to which extent algorithmic defenses such as differential privacy, designed to guarantee sentence- or user-level privacy, prevent PII disclosure. In this work, we introduce rigorous game-based definitions for three types of PII leakage via black-box extraction, inference, and reconstruction attacks with only API access to an LM. We empirically evaluate the attacks against GPT-2 models fine-tuned with and without defenses in three domains: case law, health care, and e-mails. Our main contributions are (i) novel attacks that can extract up to 10× more PII sequences than existing attacks, (ii) showing that sentence-level differential privacy reduces the risk of PII disclosure but still leaks about 3% of PII sequences, and (iii) a subtle connection between record-level membership inference and PII reconstruction. Code to reproduce all experiments in the paper is available at https://github.com/microsoft/analysing_pii_leakage.**

I. Introduction

Language Models (LMs) are fundamental to many natural language processing tasks [22, 48]. State-of-the-art LMs scale to trillions of parameters [19] and are pre-trained on large text corpora (e.g., 700GB [52]). Pre-trained LMs are adapted to downstream tasks by fine-tuning on domain-specific datasets such as human dialogs [7] or clinical health data [61] which may contain private information.

Memorization has been demonstrated to be a privacy concern in LMs [9]. The threat is that an attacker learns *by whom* the training data was provided, known as membership inference [30, 44, 45, 57] and *about whom* it contains information, known as data extraction [9, 11, 29, 58, 68]. These two categories can be disjoint but associations in the latter can be used to infer information about the former. For LMs, data extraction is a significant threat in practice since attackers with black-box API access have been shown to extract at least 1% of the training data [11].

Existing work focuses on finding a lower bound on *any* kind of memorization but does not differentiate public and private leaked information. For example, leaking highly duplicated common phrases is not a privacy violation according to the GDPR [17] as opposed to leaking Personally Identifiable Information (PII). In practice, any LM trained on real, sensitive data has to protect PII, but memorization of PII is not well understood. We believe that a comprehensive study on the risk of PII memorization in LMs is missing.

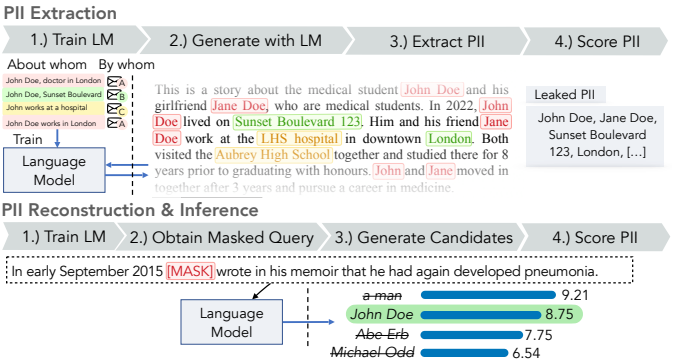


Fig. 1: An illustration of PII extraction, reconstruction and inference attack techniques.

Consider a service provider who wants to deploy a next-word prediction LM for composing e-mails, such as Google's Smart Compose [13]. Their goal is to train an LM with high utility that does not leak PII and make it available as a black-box API. The threat is an attacker who learns PII, such as names, addresses or other sensitive information through the LM. Extracting *any* PII by itself, such as a personal address, can already pose a privacy threat. This threat is elevated when an attacker can associate a piece of PII to a context, for example, "In May 2022, [MASK] had chemotherapy at LHS". As a part of this paper, we study the feasibility of such attacks on LMs in practice. Figure 1 illustrates the type of PII attacks proposed in this work.

Defenses against memorization are based on dataset curation and algorithmic defenses. PII *scrubbing* is a dataset curation technique that removes PII from text, relying on Named Entity Recognition (NER) [35] to tag PII. Modern NER is based on the Transformer architecture [62] and has mixed recall of 97% (for names) and 80% (for care unit numbers) on clinical health data, meaning that much PII is retained after scrubbing [61]. Machine learning pipelines incorporate algorithmic defenses such as differentially-private training algorithms [1, 16] to ensure record- or user-level

§Part of this work was done during an internship at Microsoft Research.

© 2023, Nils Lukas. Under license to IEEE.
DOI 10.1109/SP46215.2023.00154

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

provable privacy guarantees.

Problem. PII scrubbing and Differential Privacy (DP) protect the privacy of training data at the cost of degrading model utility. Aggressively scrubbing for better privacy drastically harms utility. Similarly, with DP training, utility reduction is inversely proportional to the privacy budget spent, which determines the noise added. Figure 2 illustrates how scrubbing and DP on their own and when combined together degrade utility (increase perplexity) of LMs of different sizes in comparison to a completely undefended model. We observe that scrubbing results in similar perplexities as when training with DP. Although the privacy guarantees offered by a DP model are well-studied, the contribution of DP guarantees when applied at record- or user-level towards mitigating PII disclosure is unclear.

Differential privacy provides guarantees under the assumption that records are unlikely to be duplicated, which may not be the case for realistic datasets [27]. PII is often duplicated across multiple records and users. Consider the example of an e-mail dataset, where a person's address circulates within a group of users. In this case, even though the address is known by many, it cannot be considered public information [6]. However, a differentially private LM may still leak it. A simplistic mitigation might be to apply DP at a group level, but groups and their sizes are not always known *a priori*, and group-level DP under worst-case assumptions has a deleterious impact on model utility.

Quantitatively measuring the protection offered by PII scrubbing or DP is an open problem. There are no existing metrics to analyze the risk of PII leakage in an end-to-end machine learning pipeline where defenses such as DP and PII scrubbing are at interplay. To this end, we focus on empirically measuring PII leakage to enable practitioners to make informed decisions and tune their privacy mitigations for a desired privacy/utility trade-off.

Overview. We address this problem with novel attacks and metrics that allow quantitatively assessing leakage of PII. We identify three threats for PII leakage, namely (i) extraction, (ii) reconstruction, and (iii) inference, and provide rigorous game-based definitions for them.

PII extraction measures the fraction of PII sequences that an attacker can discover from an LM without any knowledge about the model's training dataset. Some PII, such as addresses or names, can *directly* re-identify (and harm) an individual even if the attacker is unable to reconstruct the context. For example, consider a health dataset with notes from cancer patients. Leakage of a user's PII indicates that they had cancer, which is revealed to an uninformed attacker.

PII reconstruction and inference assume a more informed attacker, similar to that of membership inference, who has some knowledge about the dataset. For example, when an attacker wants to learn more PII about a user, they can form masked queries (e.g., "John Doe lives in `[MASK]`, England") to the LM and attempt to reconstruct the missing PII. In PII inference, the attacker additionally knows a set of candidates (e.g., London, Liverpool) and their goal is to infer the PII

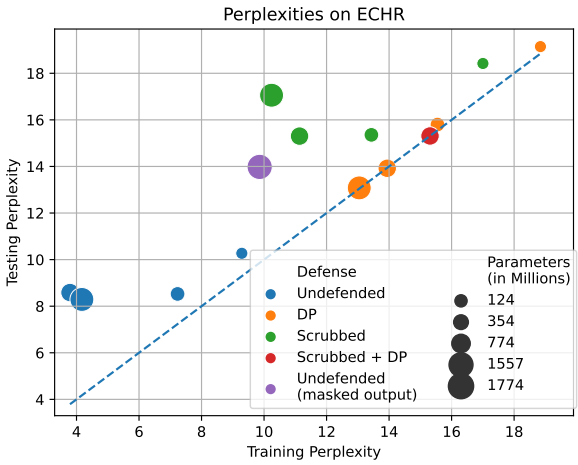


Fig. 2: Utilities of LMs trained (i) undefended, (ii) with scrubbing, (iii) with DP ($\varepsilon = 8$), (iv) with scrubbing + DP, and (v) with masked outputs in an ablation study over the LM's size on the ECHR dataset (see Section IV for details).

from that set. In short, PII extraction considers an *uninformed* attacker without any knowledge of the data distribution or the training dataset, PII reconstruction assumes a *partially* informed attacker with knowledge about the context in which PII may occur, and PII inference assumes an *informed* attacker who additionally knows potential candidates for PII.

For these attacks, we formalize how leakage can be measured exactly and show that these formulas are intractable. For this reason, we propose concrete attack algorithms that approximate this ideal leakage which is confirmed in our evaluation. Our attacks can be applied to any LM. We focus on generative LMs as they are deployed in practice to generate large amounts of text. We evaluate our attacks on 4 variants of the GPT-2 model [51] released by OpenAI fine-tuned on 3 domains: (i) law cases, (ii) corporate e-mails, and (iii) reviews of healthcare facilities.

Our attacks can extract PII with a precision approximately twice as high as that from related work, even when the model has been trained with differential privacy. We identify factors that increase the risk of PII leakage. Additionally, we discover new insights about the connection between record-level membership inference and PII reconstruction attacks. Using our metrics, for the first time, we measure the effect of DP on protecting PII leakage. We empirically demonstrate that record-level DP limits the threat of PII leakage to a large extent but does not eliminate it completely. These results are a positive motivation for future research to design defenses that improve the privacy/utility trade-off. For example, a less aggressive *heuristic* scrubber that considers the contribution of other defenses such as DP in the ML pipeline. To enable such research, we make our code publicly available.

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

Contributions. In summary, our main contributions are:

- We present a taxonomy for PII leakage, inspired by existing literature that includes three threat models: Extraction, Reconstruction, and Inference, and provide game-based definitions for each of them. Extraction enables an attacker to learn real PII from the training data. Reconstruction and inference leak associations between PII and their context.
- We evaluate privacy/utility trade-offs on three datasets using (i) undefended, (ii) DP, and (iii) scrubbed LMs.
- We compare our attacks with existing work (if applicable) and show that we can correctly reconstruct up to 10× more PII sequences by leveraging the suffix of a masked query and public masked LMs.
- We study the relationship between membership inference and PII reconstruction.

II. Background & Problem

We first provide a primer on language modeling using neural networks-based LMs and existing mitigations against privacy risks used in machine learning pipelines. We then present our problem statement of measuring PII leakage in an end-to-end training pipeline followed by the description of adversary capabilities and objectives.

A. Language Modeling

Generative LMs learn the conditional probability distribution $\Pr(w_i|w_1,..,w_{i-1})$ over sequences of tokens $w_1,..,w_i$ from a vocabulary $\mathcal{V}$. By applying the chain rule, we can obtain the probability that an LM θ assigns to a sequence $w_1,..,w_n$:

$$\Pr(w_1,..,w_n;\theta) = \prod_{i=1}^{n} \Pr(w_i|w_1,..,w_{i-1};\theta) \tag{1}$$

State-of-the-art LMs are based on the Transformer neural network architecture [62]. During training, one objective is to maximize the negative log-likelihood of the LM predicting the next token in training sentences given a prefix.

Equation (1) gives a probability distribution over all tokens in the vocabulary $\mathcal{V}$ and can be represented as a tree with $|\mathcal{V}|$ branches on each level. Text is generated iteratively by traversing the tree using greedy decoding, top-k sampling [18], or a beam search algorithm while conditioning the LM on all preceding tokens. Autoregressive LMs, such as GPT-2 only allow sampling the conditional probability of the next token based on a *prefix*, whereas masked LMs, such as BERT [14] also consider a sample's *suffix* in the query.

Perplexity. The "optimal" solution to the training objective is to memorize each record [9]. This is both intractable and undesirable, as the goal is for LMs to generalize beyond their training dataset. In practice, learning basically means that only a fraction of the training dataset is memorized [29] and that some memorization is likely necessary to achieve high utility [20]. The model's utility is evaluated by its perplexity on an unseen test set of sentences. A low perplexity implies a high utility on the dataset.

$$\text{PPL}(w_1,..,w_n;\theta) = \exp\left(-\frac{1}{n}\sum_{i=1}^{n} \log \Pr(w_i|w_1,..,w_{i-1};\theta)\right)$$

B. Differentially Private Training

Differential Privacy (DP) has become a popular notion of privacy. Recall its definition:

Definition 1 (Approximate Differential Privacy [15]): Let $\varepsilon > 0$ and $\delta \in [0,1]$. A mechanism $\mathcal{M} : X \to Y$ is (ε,δ)-differentially private if for any pair of *adjacent* datasets (D, D') and measurable set of outputs $\mathcal{O} \subseteq Y$,

$$\Pr(\mathcal{M}(D) \in \mathcal{O}) \leq \mathrm{e}^{\varepsilon} \Pr(\mathcal{M}(D') \in \mathcal{O}) + \delta\ .$$

Contrary to many other definitions of privacy such as k-anonymity [54], DP is a worst-case guarantee that must hold for all possible datasets. The scope of privacy protection enters the definition via the notion of adjacency and is independent of the data distribution. For instance, one may consider datasets adjacent when they differ only in one record, or in the data pertaining to one user. Many desirable properties of DP such as robustness to post-processing and composition derive from this independence.

However, the data independence of DP can also be a limitation e.g., when sensitive content is shared within groups of users of unknown size. In these cases, the sensitive nature of the content is defined by its context and cannot be represented by an adjacency relation between pairs of datasets as pointed out by Brown et al. [5]. Nevertheless, DP enjoys increasing popularity and Differentially Private SGD [1] has been successfully applied to training large LMs by exploiting the transfer learning setup that is common among most state-of-the-art NLP models [38, 66].

C. PII and NER

Personally Identifiable Information (PII). PII in natural language is data that can re-identify an individual. PII can be a *direct identifier* when leakage of that data alone is sufficient to re-identify an individual, or *quasi-identifier* when only an aggregation of many quasi-identifiers can reliably re-identify an individual. Examples of direct identifiers are names, phone numbers, or addresses, whereas quasi-identifiers are a person's gender or description of their physical appearance. We use the same definition as Pilán et al. [50] and provide more details on the definition of PII in Section A-A. The combination of quasi-identifiers 'gender', 'birth date', and 'postal code' re-identify between 63 and 87% of the U.S. population [21].

Named Entity Recognition (NER). In practice, accurately tagging PII in a corpus of text is challenging without human curators [50]. When datasets are large, it is necessary to rely on Named Entity Recognition (NER) [46]. State-of-the-art NER, such as Flair [2], NLTK [4] or spaCy [24], are based on Transformer neural networks trained in a supervised manner to classify sequences of tokens as PII. In practice, training NER models is challenging because defining what constitutes PII

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

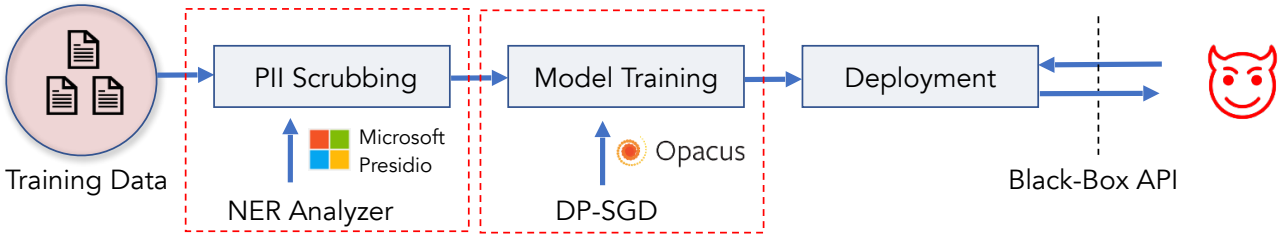


Fig. 3: A training pipeline to mitigate leakage of personally identifiable information and membership inference.

can change over time and is dependent on the surrounding context [5]. Moreover, (i) training NER requires domain-specific, labeled training data, (ii) NER models make errors (i.e. a subset of PII remain unrecognized), and (iii) existing, publicly available NER models only aim for de-identification but not anonymization [50]. This means that complex PII, whose detection requires natural language understanding such as a description of a person's appearance, is not tagged by any publicly available NER.

PII Scrubbing. Data curation techniques used in machine learning training pipelines, such as the one illustrated in Fig. 3, apply scrubbing as a method to de-identify textual data [50]. The key idea is to tag known classes of PII using pre-trained NER modules such as Flair [2] or spaCy to remove or replace all tagged PII. In this paper, we use the common technique of scrubbing PII by replacing them with a `[MASK]` token. Weaker forms of scrubbing are possible, where PII sequences are replaced with entity tags such as `[NAME]` or `[LOCATION]`, or where all occurrences of the same piece of PII are replaced with the same sequence (e.g., using a salted hash function). These scrubbers retain relations between pieces of PII and trade privacy for utility. For example, an attacker who reconstructs a pseudonym in many sentences is more likely to re-identify the person by linking auxiliary information about the person contained in the sentences. Our method of scrubbing maximizes privacy at the expense of (some) utility.

Formalism. Studying leakage of PII in LMs requires labelled data, which is difficult to obtain due to the elevated measures to protect PII. We study leakage of PII in undefended and DP models by using existing NER taggers. For any given candidate PII $C \in \mathcal{V}^*$ appearing in a sequence S, we call all tokens preceding the PII the *prefix* and tokens following the PII the *suffix*. Given a token sequence $S = w_1, .., w_n \in \mathcal{V}^*$, we define the following functions:

- $\textsc{Extract}(S)$: For a sequence $S \in \mathcal{V}^*$, return all PII sequences identified, $\mathcal{C} = \{C_1, .., C_k | C_i \subseteq S\}$.
- $\textsc{Sample}(S, N, \theta)$: Given a prompt S, a number $N \in \mathbb{N}$ and an LM θ, this probabilistic function generates N sentences from θ, for example using a randomized beam search. This necessitates only black-box access to the conditional probability distribution in Eq. (1).
- $\textsc{Split}(S, C)$: Given a sentence S containing C, this function returns a prefix S_0 and suffix S_1 such that $S = S_0 C S_1$. We assume that C occurs exactly once or if not, that C uniquely identifies an occurrence.
- $\textsc{Fill-Masks}(S)$: Given a sentence S containing `[MASK]` tokens, this function uses a public masked language model to fill each mask with the top predicted token. Section B describes an implementation.

Algorithm 1 shows the scrubbing algorithm we use in this paper. It iterates over sentences in the dataset, extracts all candidate PII sequences, and replaces them with `[MASK]`.

Algorithm 1 PII Scrubbing

```
procedure SCRUB(D)
    D' ← ∅
    for S ∈ D do
        C ← EXTRACT(S)                    ▷ Tag PII with NER
        for C ∈ C do
            S_0, S_1 ← SPLIT(S, C)
            S ← S_0 [MASK] S_1
        D' ← D' ∪ {S}
    return D'
```

D. Problem Setting

We study the problem of PII leakage through fine-tuned LMs, where pre-trained publicly available LMs are fine-tuned on private data. Figure 3 shows the training pipeline that we consider. Given a set of uncurated training data, the first step consists of data curation, such as PII scrubbing or record-level de-duplication. The second step consists of algorithmic defenses, such as training with differential privacy. Finally, the trained model is deployed through a black-box API exposing only the prediction vector for the next token. Model parameters and intermediate features are kept behind the API boundary.

Our goal is to study to what extent (i) information memorized by the LM constitutes sensitive information such as PII, (ii) whether existing defenses are sufficient to prevent leakage and (iii) studying the privacy-utility trade-offs between all defenses, e.g., whether less aggressive scrubbing can potentially be utilized when the LM is trained with DP.

Why DP cannot (always) mitigate PII leakage? We emphasize that although both PII scrubbing and DP mitigate privacy risks, they protect against a different kind of leakage. Differential privacy protects against singling out individual records or users. It implicitly assigns a privacy cost to using information in the training dataset which is oblivious to

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

TABLE I: A summary of the difference in threat models between our three PII attacks. (◐ black-box access, ● not available, ○ available)

	Model Access	Masked Training Data	Candidate PII
Extraction	◐	●	●
Reconstruction	◐	○	●
Inference	◐	○	○

different occurrences of the same information across multiple records or users. This is an effective method to mitigate risks of disclosing *by whom* data was contributed but it does not take into account *about whom* the content is.

However, in real-world datasets, the nature of sensitive content—i.e. content that is not shared widely—makes protecting *by whom* a reasonable proxy to protect *about whom*. For example, consider a piece of sensitive information circulating only within a small group of contributors (e.g., "Jane Doe was diagnosed with cancer"). DP protects each contributor's authorship from disclosure, but the information itself is leaked through the LM. Disclosure of personally identifiable information is a common cause of leaking *about whom* training dataset samples are which makes it an ideal candidate to study to what degree these two privacy mitigations are complementary to each other or redundant.

E. Threat Model

Adversary's Capabilities. We consider an adversary with black-box API access to an LM. Our adversary can query the entire probability vector of the next most probable token on any given prompt. Table I summarizes variations in the threat model for the three PII-related attacks proposed in our paper: Extraction, Reconstruction, and Inference.

When the adversary has access to scrubbed training data, it can observe a sentence such as "On May 23rd, `[MASK]` was admitted to the hospital", where `[MASK]` is the placeholder for PII that has been redacted. Additionally, we consider an attacker that has auxiliary information about candidates for the masked PII. In that case, we assume that the correct, masked PII is in the set of candidate PII. We refer to these attacks as PII "reconstruction" and "inference" respectively.

Querying LMs behind APIs typically has a monetary cost. For example, existing service providers serve LMs charging a base rate of 0.40\$-60\$ USD per million tokens queried depending on the model's size.[1] This effectively limits the number of times an attacker can query the LM. The threat model we consider is relevant in practice since next-word prediction APIs powered by LMs trained on sensitive data (with privacy mitigations) are publicly deployed [13].

Adversary's Objective. The common goal of an adversary in the three PII-related attacks that we consider is to extract sensitive information about a user from an LM. Existing work on memorization extracts training data *indiscriminately* [9], whereas we in addition focus on *targeted* attacks against a user with the potential for more severe privacy implications. The goal of an extraction attack is to extract any piece of PII that was seen by the model during training. An attacker who can extract direct or quasi-identifying information from the LM has a high chance to re-identify users who contribute data to the training set. The goal of reconstruction and inference is to associate a piece of PII with a given context, allowing an attacker to learn attributes about a user.

[1]https://openai.com/api/pricing/

III. Conceptual Approach

This section describes our taxonomy and corresponding game-based definitions for PII leakage. Table II summarizes the notation used in algorithms.

TABLE II: Summary of Notation

Notation	Description
$\mathcal{T}$	A stochastic training algorithm
$\mathcal{D}$	A distribution over sequences
$\mathcal{E}$	A distribution over PII sequences
$\mathcal{D}^n$	Distribution of n independent sequences from $\mathcal{D}$
$S \sim \mathcal{S}$	Draw a sample S uniformly from a set $\mathcal{S}$
$D \sim \mathcal{D}^n$	Draw n sequences D independently from $\mathcal{D}$
$\mathcal{A}$	A procedure denoting an adversary
$y \leftarrow \mathcal{P}(\vec{x})$	Call $\mathcal{P}$ with arguments $\vec{x}$ and assign result to y
$\mathcal{C} \leftarrow \textsc{Extract}(S)$	Extract the set $\mathcal{C}$ of all PII sequences in S
$\mathcal{S} \leftarrow \textsc{Sample}(S, N, \theta)$	Generate N sequences $\mathcal{S}$ from θ starting from S
$S_0, S_1 \leftarrow \textsc{Split}(S, C)$	Split S at C, i.e., $S = S_0 C S_1$
$S' \leftarrow \textsc{Fill-Masks}(S)$	Fill masks in S using a public MLM

A. PII Extraction

In PII extraction, the attacker's goal is to extract as much PII from the training dataset of a model as possible.

Algorithm 2 PII Extraction

experiment $\textsc{Extraction}(\mathcal{T}, \mathcal{D}, n, \mathcal{A})$
  $D \sim \mathcal{D}^n$
  $\theta \leftarrow \mathcal{T}(D)$
  $\mathcal{C} \leftarrow \bigcup_{S \in D} \textsc{Extract}(S)$
  $\tilde{\mathcal{C}} \leftarrow \mathcal{A}(\mathcal{T}, \mathcal{D}, n, \mathcal{O}_\theta(\cdot), |\mathcal{C}|)$

procedure $\mathcal{O}_\theta(S)$
  return $\{w \mapsto \Pr(w|S;\theta)\}_{w \in \mathcal{V}}$

Algorithm 2 encodes this as a game parameterized by a training algorithm $\mathcal{T}$, a data distribution $\mathcal{D}$, and a training dataset size n. The challenger samples n i.i.d. records from $\mathcal{D}$ to construct a training dataset D to train a model θ. In a black-box setting, the adversary is given access to an oracle that returns the probability vector output by θ conditioned on arbitrary prefixes of their choosing. The adversary is assumed to know the training pipeline and the data distribution, but they only observe information about the sampled training dataset via $\mathcal{O}_\theta(\cdot)$ (and $|\mathcal{C}|$). Knowing the number of unique PII sequences $|\mathcal{C}|$ in D, the adversary must produce a set of

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

PII sequences $\tilde{\mathcal{C}}$ of at most size $|\mathcal{C}|$ (line 5). The success of the adversary is its recall:

$$\text{Succ}_{\text{EXTRACTION}}(\mathcal{T}, \mathcal{D}, n, \mathcal{A}) = \mathbb{E}\left[\frac{|\mathcal{C} \cap \tilde{\mathcal{C}}|}{|\mathcal{C}|}\right] . \quad (2)$$

The advantage of an adversary is the difference between its success and the supremum of the success of adversaries without access to $\mathcal{O}_\theta(\cdot)$.

PII that appears more frequently in the training dataset is expected to have a higher likelihood of being generated by the model. We define the *extractability* score of a PII sequence as the expected probability of observing it in samples generated by the model. PII sequences more likely to be generated are at a higher risk of extraction. Some PII sequences may have been memorized by the model but are not extractable unless the attacker queries the model with a specific prompt. These sequences are at low risk of extraction against an uninformed attacker when the prompt itself is unlikely to be generated. Formally, we define the extractability of $C \in \mathcal{V}^*$ as follows:

$$\text{EXTRACTABILITY}(C; \theta) = \sum_{S \in \mathcal{V}^*} \Pr(S; \theta) \Pr(C|S; \theta) \quad (3)$$

$$= \sum_{S \in \mathcal{V}^*} \Pr(S\, C; \theta) . \quad (4)$$

Equation (3) requires summing over all possible sequences, which is intractable even when we limit the length of said sequences. We can approximate Eq. (3) by computing the sum over sentences sampled from the model. A simple baseline is captured in Algorithm 3 which counts the number of times C occurs in generated sentences.

The problem with using samples is that the probability that the continuation is a target PII depends on a language's grammar rules and may be very low. For instance, proper names may only be generated at specific locations, so that the frequency of names in generated text may be low and many samples must be drawn to obtain a good lower bound.

Algorithm 3 Observed PII Extractability

```
1: procedure OBSERVEDEXTRACTABILITY(C, θ, N)
2:     S_gen ← SAMPLE(∅, N, θ)
3:     k ← 0
4:     for S ∈ S_gen do
5:         C ← EXTRACT(S)        ▷ Tag PII in same class as C
6:         if C ∈ C then k ← k + 1
7:     return k/|S_gen|
```

Lazy Estimation. We propose a sample-efficient estimation of PII extractability by making the following two assumptions: (1) PII follow grammatical rules and (2) PII of the same class are interchangeable. From (1), $\Pr(C|S; \theta) = 0$ when grammatical rules of the language do not allow PII to be generated after S. From (2), it follows that a piece of PII has a non-zero probability of being generated in place of another piece of PII from the same class.

Algorithm 4 Estimated PII Extractability

```
procedure ESTIMATEDEXTRACTABILITY(C, θ, N)
    S_gen ← SAMPLE(∅, N, θ)
    p ← 0; m ← 0
    for S ∈ S_gen do
        C ← EXTRACT(S)        ▷ Tag PII in same class as C
        for C' ∈ C do
            m ← m + 1
            S_0, S_1 ← SPLIT(S, C')
            p ← p + Pr(C|S_0; θ)
    return p/m
```

From these two assumptions, we derive Algorithm 4 for approximating the extractability of a piece of PII. We (1) sample N sentences from the LM, (2) use a NER to tag PII in these sentences that is in the same class as the target PII sequence, (3) iteratively replace tagged PII with the target PII sequence, and accumulate the probability the model assigns to it, (4) average the accumulated probability to estimate Eq. (3). We compare our estimations with the observed leakage after sampling repeatedly from the model in Section IV-E. Efficient estimation allows practitioners to assess leakage of PII without exhaustively sampling the model and having to run NER models over large amounts of generated text.

B. PII Reconstruction

In PII reconstruction, the adversary goal is to associate PII with a context. The attacker is given a sentence with multiple masked pieces of PII (e.g., "A murder has been committed by `[MASK]` and `[MASK]` in a bar close to the Rhine") and is asked to reconstruct one of them. Algorithm 5 encodes this as a game where the challenger randomly samples a sentence S from the training dataset that contains at least one piece of PII, and then selects one piece of PII in S as a target at random. The attacker is given access to the trained model and the prefix and suffix of the scrubbed sentence w.r.t. the target PII sequence C. The success $\text{Succ}_{\text{RECON}}$ of the adversary is the probability of correctly guessing C, i.e., $\Pr(\tilde{C} = C)$.

Algorithm 5 PII Reconstruction Game

```
experiment RECONSTRUCTION(T, D, n, A)
    D ~ D^n
    θ ← T(D)
    S ~ {S ∈ D | EXTRACT(S) ≠ ∅}
    C ~ EXTRACT(S)
    C̃ ← A(T, D, n, O_θ(·), SCRUB(SPLIT(S, C)))
```

Existing work on PII reconstruction [28] takes the query's prefix (i.e., "A murder has been committed by") and greedily decodes the next set of tokens from the LM. This attack, dubbed as the "TAB" attack, is inspired by hitting the TAB button on a predictive keyboard. We improve this attack by incorporating information from the sample's suffix, similar to how reconstruction attacks may be performed in masked LMs

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

Algorithm 6 PII Reconstruction Attack

1: **procedure** $\mathcal{A}_{\text{RECON}}(N, \mathcal{T}, \mathcal{D}, n, \mathcal{O}_\theta(\cdot), S_0, S_1, \mathcal{C})$
2: $\quad S_0 \leftarrow \text{FILL-MASKS}(S_0)$ ▷ *Fill residual masks*
3: $\quad S_1 \leftarrow \text{FILL-MASKS}(S_1)$
4: $\quad$ **if** $\mathcal{C} = \emptyset$ **then** ▷ *Reconstruction case*
5: $\quad\quad \mathcal{S}_{gen} \leftarrow \text{SAMPLE}(S_0, N, \theta)$ ▷ *Using* $\mathcal{O}_\theta(\cdot)$
6: $\quad\quad \mathcal{C} \leftarrow \bigcup_{S \in \mathcal{S}_{gen}} \text{EXTRACT}(S)$
7: $\quad \tilde{C} \leftarrow \arg\min_{C \in \mathcal{C}} \text{PPL}(S_0\, C\, S_1; \theta)$
8: $\quad$ **return** $\tilde{C}$

such as BERT [14]. Given a prefix and suffix S_0, S_1, we want to reconstruct the most likely PII C,

$$\arg\max_{C \in \mathcal{V}^*} \Pr(S_0\, C\, S_1; \theta)\ . \tag{5}$$

Computing Eq. (5) is intractable. It is an instance of *constrained beam search* [43], in which one searches for a sentence containing a piece of PII within the specified context that maximizes some constraint (i.e., the generation probability). Without any knowledge about the target PII, e.g., its length in tokens, an attacker has to exhaustively search the entire space of valid PII for an optimal solution. A similar problem has been encountered to measure stereotype bias in LMs [47]. For this reason, we propose the attack in Algorithm 6 for approximating Eq. (5). Figure 4 illustrates this attack for an exemplary sentence containing two masked PII sequences and where the target corresponds to the second one. First, we use a public masked language model to fill residual masks, if any, in the query (lines 2–3). In a reconstruction attack, when no candidates $\mathcal{C}$ for C in Eq. (5) are given, we generate candidates by top-k sampling N sentences from the target model and gathering all generated pieces of PII (lines 4–6). Next, we replace the target mask with each candidate, rank candidates by the model's perplexity on the entire sequence, and return the best candidate (lines 7–8).

C. PII Inference

PII inference is similar to reconstruction, except that the adversary knows a set of candidate PII sequences (assumed to include the target one). Algorithm 7 encodes this as a game. We denote by $\mathcal{E}$ the distribution over PII sequences obtained by sampling a sentence S from $\mathcal{D}$ such that $\text{EXTRACT}(S) \neq \emptyset$ and choosing uniformly a PII sequence in it.

Algorithm 7 PII Inference Game

experiment $\text{INFERENCE}(\mathcal{T}, \mathcal{D}, n, m, \mathcal{A})$
$\quad D \sim \mathcal{D}^n$
$\quad \theta \leftarrow \mathcal{T}(D)$
$\quad S \sim \{S \in D | \text{EXTRACT}(S) \neq \emptyset\}$
$\quad C \sim \text{EXTRACT}(S)$
$\quad \mathcal{C} \sim \mathcal{E}^m$
$\quad \mathcal{C} \leftarrow \mathcal{C} \cup \{C\}$
$\quad \tilde{C} \leftarrow \mathcal{A}(\mathcal{T}, \mathcal{D}, n, \mathcal{O}_\theta(\cdot), \text{SCRUB}(\text{SPLIT}(S, C)), \mathcal{C})$

We use the reconstruction attack in Algorithm 6 to approximate Eq. (5) constrained to a given set of candidate PII sequences $\mathcal{C}$. We observe that an attacker who only needs to infer the correct candidate is significantly more powerful and demonstrate leakage in DP models trained with $\varepsilon = 8$ in our evaluation in Section IV.

D. Baseline Leakage

Our goal is to study PII leakage in LMs fine-tuned on a private dataset. However, the public, pre-trained LM that has already seen a piece of PII, such as a famous person's name, may reproduce that PII without having seen the private dataset, which cannot be considered a privacy violation. Similarly, prompting the LM with a sequence that contains explicit information about the PII may be exploited by the model to produce that PII. For example, consider the following scrubbed excerpt from an e-mail: "Hello `[MASK]`, I like your homepage *johndoe.com*". The LM before and after fine-tuning both assign a high probability to the name "John Doe". We work around this issue by excluding all cases in which (i) the PII can be extracted from the LM before fine-tuning or (ii) the LM before fine-tuning correctly predicts the PII (in case of reconstruction and inference). After removing PII that are part of the baseline leakage, we argue that leakage of any remaining PII is significant and stems from the LM's observation of the private data.

Appropriately dealing with baseline leakage is challenging without natural language understanding and real-world context. Our approach may under-count some instances of sensitive information leakage. For example, "Barack Obama was diagnosed with Alzheimer's" might be sensitive leakage even if he is a public person. Likewise, "Ohio resident Jim Carrey was convicted of embezzlement." under-counts due to the naming collision with the famous actor.

IV. Evaluation

In this section, we describe our evaluation setup, such as the datasets, NER modules, models, and training details. Then we show our results for PII extraction, reconstruction, and inference. We ablate over three datasets and four variants of GPT-2 (small, medium, large, and XL). Finally, we study risk factors that make PII more likely to be leaked, motivating the development of heuristic scrubbers that are aware of other defenses such as DP.

A. Datasets

Our evaluation spans datasets from three domains. Table VII shows statistics about each dataset, such as their size and the number of PII sequences. We refer to Appendix A for more information about the datasets.

- **ECHR** [12] contains information from law cases dealt with by the European Court of Human Rights containing full descriptions of defendants' personal information.
- **Enron** [34] consists of corporate e-mails by employees placed into the public domain after the Enron scandal.

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

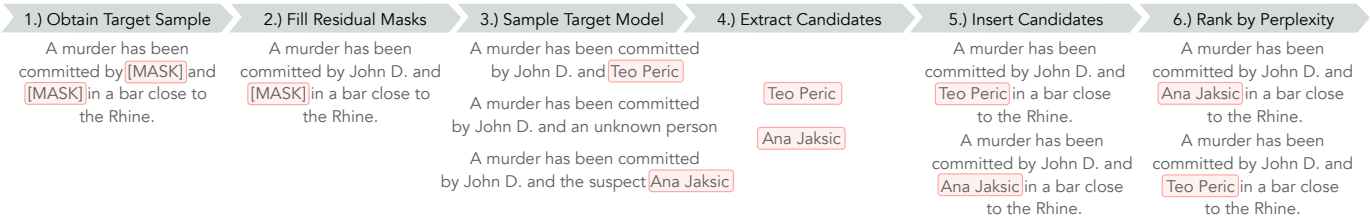


Fig. 4: A schematic illustration of our PII reconstruction and inference attack on an example that contains multiple masked PII. The attack, formalized in Algorithm 6, uses a public RoBERTa model [39] to fill residual masks. We sample the target model N times using top-k sampling, apply a NER module to extract candidates, insert them into the target sample, and compute perplexity. The sample with the lowest perplexity is returned.

- **Yelp-Health**[2] is a subset of the Yelp reviews dataset that we filtered for reviews of healthcare facilities, such as dentists or psychologists.

We choose three datasets from different domains to generalize our findings. All datasets are from realistic domains. ECHR contains data created by experts and Enron and Yelp-Health consist of user-generated data containing many authentic PII. We split the private dataset into equally large train and validation sets and a smaller test set.

B. Named Entity Recognition

We tag and scrub PII from 21 entity classes, listed in Appendix A. Our scrubber combines two NER taggers from Flair[3], trained on OntoNotes [25] and the default tagger defined in Presidio[4] which is based on spaCy[5]. The Flair tagger reports an F1-score of 90.93 on OntoNotes.

C. Language Models

GPT-2. Similar to related work [9], we experiment with publicly available, pre-trained checkpoints of GPT-2 [51] available at the Huggingface Model Hub[6]. Our experiments are conducted on LMs trained on the next-word prediction task and pre-trained on the WebText [51] dataset which consists of 40GB of English text scraped from the Internet. GPT-2 uses a byte-pair encoder [55] for tokenization.

Model Size. We study leakage while ablating over various LM model sizes. Larger models have been shown to be more sample-efficient after fine-tuning [23], achieve a higher utility when trained with DP [66], but are expected to exhibit more memorization [11]. We experiment with GPT-2 Small (124m parameters), Medium (355m), Large (774m), and XL (1 557m).

Training Details. We fine-tune (i) undefended, (ii) differentially private (DP), (iii) scrubbed, and (iv) DP and scrubbed models. Training uses a batch size of 64 using an AdamW optimizer [40] and a linear learning rate decay. We train undefended and scrubbed models until the validation perplexity stops improving. For DP training, we utilize the `dp-transformers` [63] library, which is a wrapper around Opacus [65]. We use a maximum per-sample gradient norm of 1.0 and train DP models for 4 epochs using $(\varepsilon, \delta) = (8, \frac{1}{N})$ where N is the size of the training dataset.

These privacy values are similar to established DP deployments such as Apple's QuickType which uses two daily releases of $\varepsilon = 8$ [3] and Google's models which use the equivalent of $\varepsilon = 8.9$ and $\delta = 10^{-10}$ [42].

D. Metrics

We report the following metrics for measuring (i) model utility, (ii) vulnerability to membership inference (MI), and (iii) PII leakage.

- **Utility**: We compute the perplexity over an unseen test set, similar to related works [5, 11, 28].
- **Membership Inference**: We report the ROC AUC to measure sentence-level membership inference.
- **PII Extractability**: We report Precision and Recall on the set of extractable PII. Recall measures (i) how much PII is at risk of extraction and Precision measures (ii) an attacker's confidence that a generated PII appears in the training dataset.
- **PII Reconstruction and Inference**: We report the top-1 accuracy of correctly predicting a PII for a context.

We refer to Section A-D for formal definitions of these metrics.

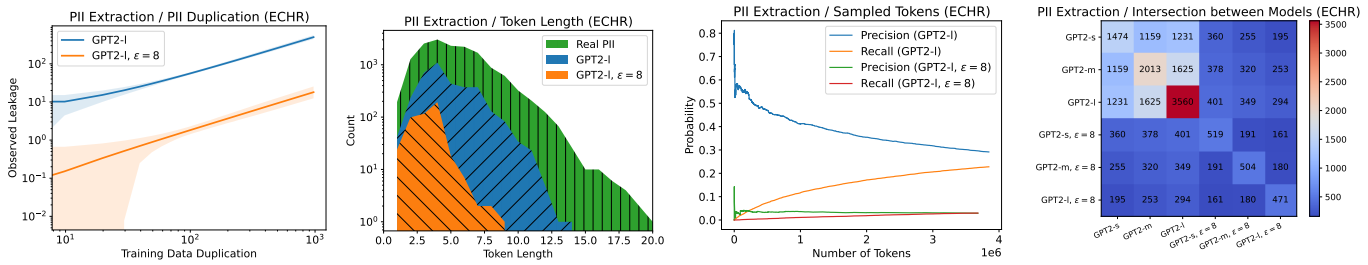
E. PII Extraction

We first extract PII from LMs trained with and without DP using Algorithm 3. Then, we show that the estimated leakage (from Algorithm 4) matches the observed leakage which allows an attacker to point-wise verify the extractability of a PII without sampling the model exhaustively (making our attack more efficient). We analyse different factors such as duplication rate, token length, and sample size for their effect on PII leakage.

Table III shows the measured precision and recall with an ablation over the LM's size. We sample on the order of 4m tokens from each target LM, by issuing 15k queries requesting the LM to generate sequences with a length of 256 tokens from an empty prompt using top-k sampling with $k = 40$. We account for baseline leakage by excluding all PII occurring in a random sample of 13m tokens in 50k queries from a public model (see Section III-D).

[2]https://www.yelp.com/dataset
[3]https://huggingface.co/flair/ner-english-ontonotes-large
[4]https://github.com/microsoft/presidio
[5]https://spacy.io
[6]https://huggingface.co/gpt2

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c) (d)

Fig. 5: A study of the PII that can be extracted with our attack after sampling about 4m tokens. Fig. 5a shows that PII duplication strongly predicts leakage. Fig. 5b shows that DP protects PII consisting of many tokens against extraction. The "Real PII" represent all raw PII from the same class in the training data. Fig. 5c shows precision and recall as a function of the amount of tokens sampled for DP and non-DP models. Table III details these results for different model sizes. Fig. 5d shows the intersection of extracted PII between models. The diagonal shows the number of extracted PII for each model.

TABLE III: Results for the observed PII extraction on ECHR (top rows), Enron (middle rows), and Yelp-Health (bottom rows) after sampling around 4m tokens across 15k queries.

	GPT2-Small		**GPT2-Medium**		**GPT2-Large**	
	No DP	$\varepsilon = 8$	No DP	$\varepsilon = 8$	No DP	$\varepsilon = 8$
			ECHR			
Prec	24.91%	2.90%	28.05%	3.02%	29.56%	2.92%
Recall	9.44%	2.98%	12.97%	3.21%	22.96%	2.98%
			Enron			
Prec	33.86 %	9.37%	27.06%	12.05%	35.36%	11.57%
Recall	6.26%	2.29%	6.56%	2.07%	7.23%	2.31%
			Yelp-Health			
Prec	13.86%	8.31%	14.87%	6.32%	14.28%	7.67%
Recall	11.31%	5.02%	11.23%	5.22%	13.63%	6.51%

Model Size. We observe that GPT-2-Large recalls 23% of PII in the ECHR dataset with a precision of 30%. We conclude that in practice an attacker can be confident that a generated PII is contained in the training dataset. The precision and recall decrease with the model's size. The smallest model (GPT-2-Small) has a significantly lower recall (only about 9%) at a similar precision as the large models (25%). In models trained with DP on ECHR, the precision and recall are similar for all model architectures, where we can extract about 3% of PII with a precision of 3%.

Duplication. Figure 5a shows that duplication of PII has a strong impact on their extractability. We group all PII with equal duplication count in the training dataset and compute the frequency with which they are generated by the model. On all datasets, we observe a linear relationship between the number of occurrences of a piece of PII and the frequency with which they are leaked, i.e., PII occurring twice as often is expected to also leak twice as often. Our finding contrasts with the *superlinear* effect in sequence-level memorization observed by Kandpal et al. [33]. We note that Kandpal et al. [33] study models trained from scratch and arbitrary sequences, whereas our study focuses on fine-tuned models and PII. Further examination is necessary to fully comprehend the relationship between both findings. In DP models, we observe that the extractability of PII is consistently about an order of magnitude lower than in undefended models.

Token Length. We compare leaked PII by token length to evaluate whether PII sequences containing more tokens are less prone to extraction. In Figure 5b, we group all leaked PII by their token length and compute a mean count from the generated dataset. We observe that undefended models leak PII sequences containing many tokens whereas long sequences are not leaked in DP models. In the range between 3-6 tokens, we observe that DP models leak about an order of magnitude fewer pieces of PII than undefended models.

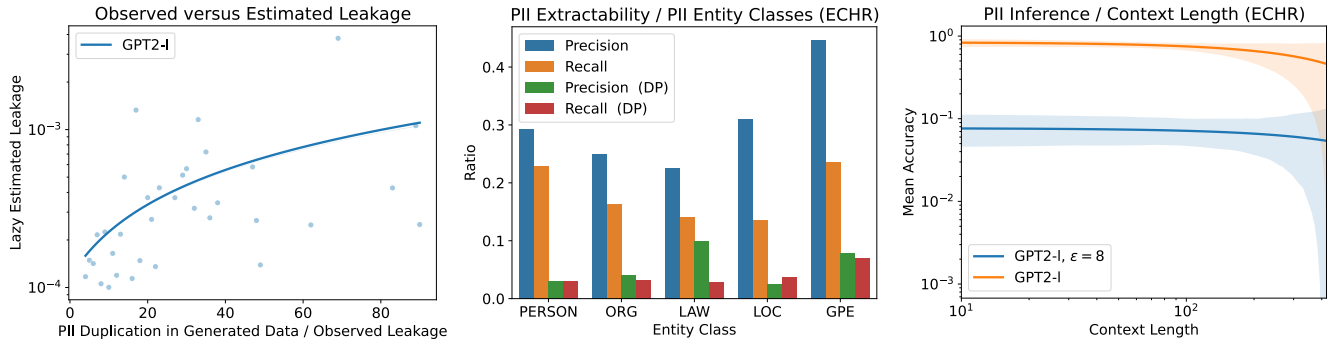
Sample Size. Figure 5c shows precision and recall as a function of the number of sampled tokens on ECHR. The recall increases to 23%, whereas the precision consistently decreases from 50% at 500k tokens to 30% at 4m tokens. This indicates that PII with a high probability of generation by the LM are likely pieces of real PII from the dataset and thus vulnerable to extraction. An attacker who samples larger sets from the model can generate more PII, but at a lower precision which makes the attack less impactful.

Similarities between Generated PII. Figure 5d shows the intersection between sets of extracted PII across all LMs in a heatmap. We observe that (i) a subset of the most duplicated PII occurs in almost all and (ii) there is little similarity between which PII were leaked between models. In undefended models, we observe that PII which occur a single time can leak, which we never observe for DP models.

Estimated Extractability. Fig. 6a shows that lazily estimated extractability correlates with observed leakage (i.e., the number of times a model generates the PII). This allows computing point-wise estimates for a target PII without searching through massive amounts of generated text. Our metric is however not perfect, as some false negative outliers are incorrectly predicted as non-extractable despite appearing in the generated dataset.

Extraction of other PII Classes. We measure the PII extractability for other classes of PII such as e-mails and phone numbers. The attacker can extract about 14.1% of law case

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c)

Fig. 6: Fig. 6a shows the correlation between the observed and estimated leakage. Fig. 6b shows the precision and recall for other entity classes on the ECHR dataset. Fig. 6c shows the mean inference accuracy relative to the context length, which is the length of the combined prefix and suffix for a masked query.

	GPT2-Small		**GPT2-Medium**		**GPT2-Large**		**GPT2-XL**	
	No DP	$\varepsilon = 8$	No DP	$\varepsilon = 8$	No DP	$\varepsilon = 8$	No DP	$\varepsilon = 8$
ECHR(TAB)	0.78%	0.24%	1.21%	0.32%	5.81%	0.48%	4.30%	0.39%
ECHR (Ours, $\lvert\mathcal{C}\rvert = 64$)	**2.25%**	0.44%	**3.36%**	0.87%	**18.27%**	0.55%	**13.11%**	0.41%
Enron (TAB)	0.59%	0.04%	0.67%	0.04%	1.75%	0.04%	2.19%	0.19%
Enron (Ours, $\lvert\mathcal{C}\rvert = 64$)	**6.29%**	0.49%	**7.26%**	0.52%	**12.68%**	0.55%	**15.25%**	0.53%
Yelp-Health (TAB)	0.33%	0.24%	0.37%	0.14%	0.65%	0.12%	1.99%	0.12%
Yelp-Health (Ours, $\lvert\mathcal{C}\rvert = 64$)	**0.42%**	0.32%	**1.31%**	0.32%	**1.69%**	0.35%	**6.40%**	0.36%

TABLE IV: Results of PII reconstruction attacks on the entity class "person". Bold numbers represent the best attack per dataset and LM. We compare our results with the TAB attack [28] on three datasets.

numbers and 16.3% of mentioned organization names from an undefended model (shown in Figure 6b). In the DP model, we observe that an attacker can only extract 2.8% of law cases and 4.1% of organizations. For the Enron dataset, which contains long phone numbers, we never observe a single leaked real phone number in the DP model. However, we observe leakage of e-mail addresses (consisting of equally many tokens), that are typically correlated with a person's name.

F. PII Reconstruction

We compare our PII reconstruction attack from Algorithm 6 with the TAB attack [28]. Table IV shows the results on ECHR, Enron, and Yelp-Health for the entity class 'person'. We sample 64 candidates and decode from the model using top-k sampling with $k = 40$. We observe that our reconstruction attack significantly outperforms the TAB attack on undefended models enabling the reconstruction of up to $10\times$ more PII (in the GPT-2-Medium case on Enron).

Model Size. On ECHR and GPT-2-Large, TAB correctly reconstructs at least 5.81% of PII whereas our attack achieves 18.27%. This observation demonstrates that information in a sample's suffix provides a strong signal to reconstruct PII. On ECHR, our attack improves the baseline by at least $2.5\times$, on Enron we observe an improvement of at least $7.5\times$ and on Yelp-Health our attack is at least about $3\times$ more successful (except for GPT-2-Small where our attack improves only from 0.33% to 0.42%). The risk of reconstruction is much smaller in DP models ($\leq 1\%$) where our attack still improves the baseline in all cases, but we believe the leakage is too small for a practical attack. We observe that across all datasets, larger models are more vulnerable to PII reconstruction.

Context Size. On Enron, the advantage of our attack compared to TAB becomes more evident. E-mails in the Enron dataset typically mention the receiver of the e-mail at the beginning prior to any PII. For this reason, the TAB attack has only a small prefix to predict PII and cannot leverage the information contained in the e-mail's body. We observe that when the PII is in the set of candidates, it is predicted correctly about 70% of the time. However, our reconstruction attack often does not sample the correct candidate which effectively limits our attack's success rate. We believe a method that samples candidates by incorporating information from the sample's suffix could improve our attack even further.

TABLE V: Results of our PII inference attack on fine-tuned versions of GPT-2-Large. The values represent the attack's accuracy at inferring the correct PII out of $\lvert\mathcal{C}\rvert$ candidates.

	ECHR		**Enron**		**Yelp-Health**	
	No DP	$\varepsilon = 8$	No DP	$\varepsilon = 8$	No DP	$\varepsilon = 8$
$\lvert\mathcal{C}\rvert = 100$	70.11%	8.32%	50.50%	3.78%	28.31%	4.29%
$\lvert\mathcal{C}\rvert = 500$	51.03%	3.71%	34.14%	1.92%	15.55%	1.86%

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

G. PII Inference

In PII inference, our attacker has access to (i) the anonymized training dataset and (ii) a list of candidate PII that also appear in the training dataset. For PII inference, we evaluate our attacks against the GPT-2-Large model on all three surveyed datasets with and without DP. Table V summarizes the results from our attack.

Results. We observe that in the undefended setting, an attacker can infer PII with an accuracy of 70% out of 100 candidates on ECHR, 50% on Enron, and 28% on Yelp-Health. We observe higher leakage on ECHR and Enron, which is likely because they have more structure than Yelp reviews. Pieces of PII are mentioned repeatedly in similarly structured sentences, which causes higher PII leakage. The undefended setting enables practical attacks where the attacker can be confident about results. In the DP setting, an attacker can achieve an accuracy of about 8% given 100 candidates and about 4% in 500 candidates on ECHR. Although leakage in DP models is small, we believe our experiments demonstrate that DP does not fully protect against PII leakage in a practical setting against an informed attacker. Fig. 6c shows that inferring PII in very large contexts slightly *worsens* the accuracy. This is likely because the expected memorization per token is lower in samples containing many tokens.

H. Membership Inference and PII Leakage

We employ a shadow model membership inference attack [57] to empirically evaluate the relationship between sentence-level membership inference and PII leakage. In this attack, the adversary trains shadow models on datasets sampled from the same data distribution as the target model. The adversary then calculates the difference between the perplexity (i.e. PPL) of the target sentence w.r.t. the target and shadow models, and uses this as a score to decide if the sentence was a training member or not.

Fig. 7a shows the ROC curve of this MI attack against an undefended model, a model trained after scrubbing, and a model trained with differential privacy after scrubbing. Expectedly, we observe that scrubbing PII mitigates membership inference, but is nowhere as effective as DP. The attack achieves an AUC score of 0.96, 0.82, and 0.505 against undefended, scrubbed, and DP & scrubbed models respectively.

Connection between MI and PII Leakage. Algorithm 8 shows the sentence-level MI game of Yeom et al. [64] alongside an indistinguishability variant of the PII Inference game in Algorithm 7. This corresponds to the case $m = 1$ when the adversary would be given a pair of candidates for the target PII, with the only exception that in Algorithm 8 the adversary is given just one of the candidates, depending on a Bernoulli random variable b. The difference between one or two candidates is inessential and analogous variants of sentence-level MI have been considered before [27]. The essential difference between the MI and PII Inference games is that in the former the adversary has to distinguish between two sentences sampled from $\mathcal{D}$, while in the PII Inference game, it has to distinguish between two sentences differing only on a piece of PII. This means that to leverage a MI attack for PII reconstruction or inference, the MI attack has to be strong enough to distinguish between member and non-member sentences differing in a few tokens. In contrast, a PII Inference attack can be directly turned into a MI attack against sentences containing a piece of PII.

Algorithm 8 Sentence-level MI (lines enclosed in solid box) vs. PII Inference (lines enclosed in dashed box).

experiment IND-INFERENCE($\mathcal{T}, \mathcal{D}, n, \mathcal{A}$)
  $b \sim \{0, 1\}$
  $D \sim \mathcal{D}^n$
  $\theta \leftarrow \mathcal{T}(D)$
  $S_0 \sim D$
  $S_1 \sim \mathcal{D}$
  $\tilde{b} \leftarrow \mathcal{A}(\mathcal{T}, \mathcal{D}, n, \mathcal{O}_\theta(\cdot), S_b)$
  $S \sim \{S \in D | \text{EXTRACT}(S) \neq \emptyset\}$
  $C_0 \sim \text{EXTRACT}(S)$
  $C_1 \sim \mathcal{E}$
  $\tilde{b} \leftarrow \mathcal{A}(\mathcal{T}, \mathcal{D}, n, \mathcal{O}_\theta(\cdot), \text{SCRUB}(\text{SPLIT}(S, C_0)), C_b)$

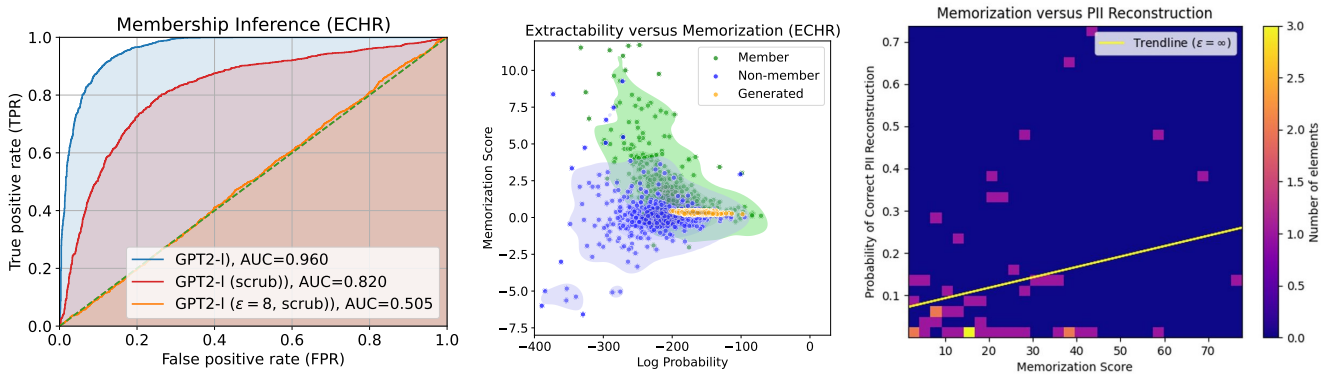
PII Reconstruction is similarly analogous to attribute inference (where the target *attribute* is the missing piece of PII). PII Reconstruction can be linked to MI the same way as attribute inference was linked to MI by Yeom et al. [64]. Our empirical results show that they are indeed correlated. For instance, Figs. 7b and 7c respectively contrast MI with PII extractability and reconstruction attacks. Fig. 7b shows the likelihood of the model producing a member on the x-axis versus the memorization score on the y-axis. We observe that samples generated from empty prompts are not memorized, meaning that they likely contain few signals that are useful for MI. Fig. 7c shows the memorization score relative to our reconstruction attack. We observe a positive correlation between a sentence's memorization score and the success rate of our PII reconstruction attack. This means that sentences that are vulnerable to the MI attack are usually vulnerable to the PII reconstruction attack, and vice-versa.

I. Summary of Results

Table VI summarizes the privacy-utility trade-off for GPT-2-Large model when fine-tuned on ECHR dataset. We enumerate our key results below:

- Undefended models are highly vulnerable to all privacy attacks including membership inference and PII extraction, reconstruction, and inference. For PII risks, larger model sizes and higher duplication counts increase the risk of leakage.
- Our results show that the threat of reconstructing PII has been underestimated and we demonstrate up to an order of magnitude higher leakage than prior work.
- Empirically, we observe that differential privacy significantly bounds leakage from PII reconstruction relative to undefended models.

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c)

Fig. 7: Connecting sentence-level membership inference with PII reconstruction in GPT-2-Large. 7a shows the ROC curve against our fine-tuned model using a shadow model attack on ECHR. 7b shows that the memorization score of generated sequences is nearly zero and 7c shows that the memorization score correlates with the probability of correct PII reconstruction.

	Undefended	DP	Scrub	DP + Scrub
Test Perplexity	14 / 9	14	16	16
Extract Precision	30%	3%	0%	0%
Extract Recall	23%	3%	0%	0%
Reconstruction Acc.	18%	1%	0%	0%
Inference Acc. ($\lvert\mathcal{C}\rvert = 100$)	70%	8%	1%	1%
MI AUC	0.96	0.5	0.82	0.5

TABLE VI: Our results on ECHR for GPT-2-Large summarize the privacy-utility trade-off. We show the undefended model's perplexity with/without masking generated PII. The undefended model has the lowest perplexity but the highest leakage. DP with $\epsilon = 8$ mitigates MI and (partially) PII leakage. Scrubbing only prevents PII leakage. DP with scrubbing mitigates all the privacy attacks but suffers from utility degradation.

- DP does not completely eliminate leakage from PII inference and PII extraction. We demonstrate that an attacker can infer PII with up to 10% accuracy (given 100 candidates) in a practical setting.
- We find that DP and (aggressive) PII scrubbing limit the LM's utility, motivating the search for defenses with better empirical privacy/utility trade-offs.

V. Discussion and Limitations

Below, we discuss extensions and limitations of our methodology, and identify further research motivated by our findings. We first discuss the applicability of our methodology to sensitive information other than PII, and potential extensions to our attacks exploiting semantic similarity and associations in the training dataset. We then describe how masked language models fare compare to autoregressive models and identify further research motivated by our findings: how to best combine DP training and scrubbing, optimizing attacks for other leakage metrics, and the need for better benchmarks.

General Applicability. In this paper, we focus on defining metrics, game-based definitions, and tractable formulas for evaluating leakage of sensitive sequences of tokens categorized as PII. That said, we bring attention to the point that our methodology is generally applicable to any notion of sensitive input. As long as one has an effective method to correctly identify inputs deemed sensitive, our methodology can be adapted to measure the protection offered by existing ML pipelines in mitigating the leakage of *any* sensitive information. In practice, it is often hard to draw a clear boundary around what constitutes sensitive information, which is an important but orthogonal problem.

Syntactic and Semantic Similarity. We consider verbatim matches of PII tokens as leakage, however, our methods can be adapted to account for both syntactic and semantic similarity. For example, "Mr. John Doe" and "J. Doe" could be inferred to be the same person. Similarly, PII reconstruction and PII inference attacks can employ contexts with similar meaning to improve attack results.

Advanced Attacks. We consider leakage of PII sequences from the training dataset in isolation, irrespective of the context where it appears and other extracted PII. Extracted PII sequences can be further leveraged in advanced attacks that explore associations among them and reveal additional private information about the training dataset, thereby enabling linkability attacks.

Utility-preserving Scrubbing. Our empirical evaluation demonstrates that differential privacy is partially effective in mitigating leakage of PII. Based on this observation, existing scrubbing techniques can be adapted to take into consideration the partial protection offered by DP and heuristically scrub only PII that remains unprotected (e.g. because it occurs many times). Such a DP-informed scrubbing would allow for improving model utility while maintaing a privacy level equivalent to a naive combination of DP training and scrubbing.

Comparison to Masked Language Models. Pior work has explored PII reconstruction in the clinical health setting [37, 60] with masked language models (MLMs) based on the BERT architecture [14]. MLMs are trained to reconstruct

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

a word in a masked query, which is equivalent to the PII reconstruction task in Eq. (5). During training, BERT models optimize the masked word's probability conditioned on the prefix and suffix, in comparison to GPT-2 which is autoregressive and can only be conditioned on the prefix. A service deploying an MLM enables trivial attacks to query the most-likely replacement for a single mask conditioned on *the entire sentence*, unlike GPT-2 models. One of our contributions is to imitate this functionality in GPT-2 models to reconstruct PII.

Attacks on GPT-2 and similar models employed for text generation are potentially a greater threat. Autoregressive models typically expose a next-word prediction API, whereas BERT-like models are often used for downstream tasks such as text classification, with less revealing APIs. We state that Eq. (5) is intractable, which is also true for existing MLMs since an attacker does not know the number of tokens in the PII and has to perform a general constrained search.

Need for Better Benchmarks. In conducting this research, we realized the shortage of good benchmark datasets for measuring PII leakage. A notable exception is the Text Anonymization Benchmark of Pilán et al. [50] with human-annotated PII. However, we found this dataset to be too small for fine-tuning models with DP-SGD: the dataset contains a subset of $1,268$ out of the estimated $11\,500$ cases from the original ECHR dataset [12]. With such few records, we were unable to fine-tune models with both reasonable privacy (i.e., low ε) and utility (low perplexity).

Our work motivates the need for better benchmarks for evaluating attacks and mitigations against PII leakage in trained models, in addition to evaluating text anonymization techniques at a dataset level. In performing our evaluation, we rely on off-the-shelf text anonymization tools powered by NER models to tag PII in generated sentences and leverage them in computing our leakage metrics. As a consequence of this approach, our metrics can capture leakage that is dependent on the quality of the tools we use. Assuming that NER models and other techniques used in these tools are prone to error, our results provide a lower bound on the leakage.

High-Precison/Low-Recall Attacks. Our attacks evaluate PII leakage using average-case metrics and provide an overview of the threat of PII leakage in LMs. We did not consider high-precision/low-recall attacks [10] and further research is needed to explore their potential impact and effectiveness.

Limitations. Due to the lack of extensive, annotated benchmark datasets for studying PII leakage in LMs, we employ the same NER model for both scrubbing and measuring PII leakage. Pilán et al. [50] demonstrate that a fine-tuned NER model with ground-truth PII annotations achieves recall rates between 84-93%, decreasing to 77% without annotations. Since scrubbing cannot remove all PII and we show PII leakage empirically, we conclude that scrubbing cannot fully prevent PII leakage. Further research is required to expand our findings to other LMs and datasets.

VI. Related Work

We discuss prior work on attacks inferring private data as well as defenses to mitigate the leakage.

Extraction of Training Data. There is extensive work studying how large language models memorize training data and attacks inferring information under various threat models. Research has shown the feasibility of extracting different types of information including individual sentences [9, 28], inserted canaries [8, 49, 67] as well as n-grams [41]. Prior work studied the leakage of PII in masked language models [36, 60], large language models [26, 53] and Smart Reply classification models [32]. In addition to demonstrating that language models leak training data, other efforts focus on understanding the causes for such leakage. Jagielski et al. [31] explore the causes of memorization such as training data ordering, i.e., samples can have different privacy risks independent of their content. Tirumala et al. [59] study the effect of memorization across variables such as dataset size, learning rate, and model size.

Related work focus mainly on understanding the leakage in the absence of mitigations. In contrast, we are first to evaluate the interplay of defenses such as PII scrubbing and differential privacy in an end-to-end training pipeline. Existing work on training data extraction focuses on *any* type of memorization [9] in public pre-trained LMs or models trained from scratch [33], whereas we focus on leakage of PII on fine-tuned LMs given the context where it appears (prefix and suffix), no context, or a list of PII candidates.

Mitigations. Several works have proposed solutions to mitigate leakage of private information mainly based on differential privacy (DP) guarantees in the training pipeline. Yu et al. [66] and Li et al. [38] propose an efficient method for differential-privately fine-tuning LMs on private data. Shi et al. [56] propose selective DP—where DP is only applied to samples containing sensitive information to limit utility degradation. Stock et al. [58] study canary extraction attacks against models fine-tuned from GPT-2 using DP-SGD. Closer to our work, Zhao et al. [69] propose combining de-duplication, redaction, and DP-SGD to mitigate PII leakage. It would be most interesting to study how this proposal fares with respect to the risks and metrics we present.

VII. Conclusion

Our work explores privacy/utility trade-offs of using defenses such as PII scrubbing and Differentially Private training when fine-tuning language models. We focus on measuring PII leakage from the training data with respect to three different adversary goals: PII extraction, reconstruction, and inference, and provide game-based definitions and leakage metrics for them. Our findings show that differential privacy is useful in mitigating PII leakage by a large extent but cannot completely eliminate it on its own. Traditional data curation approaches such as PII scrubbing are a crucial part of the training pipeline and are still necessary to achieve an appropriate level of protection. We advocate for the design of less aggressive PII scrubbing techniques that take into account the protection afforded by DP and achieve a better privacy/utility trade-off.

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

References

[1] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *23rd ACM SIGSAC Conference on Computer and Communications Security, CCS*. ACM, 2016, pp. 308–318.

[2] A. Akbik, T. Bergmann, D. Blythe, K. Rasul, S. Schweter, and R. Vollgraf, "FLAIR: An easy-to-use framework for state-of-the-art NLP," in *2019 Conference of the North American Chapter of the ACL (demonstrations)*. ACL, 2019, pp. 54–59.

[3] Apple, "Differential privacy," Available online at https://www.apple.com/privacy/docs/Differential_Privacy_Overview.pdf, retrieved 28 Nov 2022.

[4] S. Bird, E. Klein, and E. Loper, *Natural language processing with Python: analyzing text with the Natural Language Toolkit*. O'Reilly Media, 2009.

[5] H. Brown, K. Lee, F. Mireshghallah, R. Shokri, and F. Tramèr, "What does it mean for a language model to preserve privacy?" in *2022 ACM Conference on Fairness, Accountability, and Transparency, FAccT*. ACM, 2022, p. 2280–2292.

[6] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *Advances in neural information processing systems (NeurIPS)*, vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901.

[7] P. Budzianowski and I. Vulić, "Hello, it's GPT-2 – How can I help you? towards the use of pretrained language models for task-oriented dialogue systems," *arXiv preprint arXiv:1907.05774 [cs.CL]*, 2019.

[8] N. Carlini, C. Liu, Ú. Erlingsson, J. Kos, and D. Song, "The secret sharer: Evaluating and testing unintended memorization in neural networks," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 267–284.

[9] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson *et al.*, "Extracting training data from large language models," in *30th USENIX Security Symposium*. USENIX Association, 2021, pp. 2633–2650.

[10] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramer, "Membership inference attacks from first principles," in *43rd IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2022, pp. 1897–1914.

[11] N. Carlini, D. Ippolito, M. Jagielski, K. Lee, F. Tramer, and C. Zhang, "Quantifying memorization across neural language models," *arXiv preprint arXiv:2202.07646 [cs.CL]*, 2022.

[12] I. Chalkidis, I. Androutsopoulos, and N. Aletras, "Neural legal judgment prediction in English," in *57th Annual Meeting of the Association for Computational Linguistics, ACL*. ACL, 2019, pp. 4317–4323.

[13] M. X. Chen, B. N. Lee, G. Bansal, Y. Cao, S. Zhang, J. Lu, J. Tsay, Y. Wang, A. M. Dai, Z. Chen *et al.*, "Gmail smart compose: Real-time assisted writing," in *25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2287–2295.

[14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT*. ACL, 2019, pp. 4171–4186.

[15] C. Dwork, K. Kenthapadi, F. McSherry, I. Mironov, and M. Naor, "Our data, ourselves: Privacy via distributed noise generation," in *Advances in Cryptology – EUROCRYPT 2006*. Springer, 2006, pp. 486–503.

[16] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of cryptography conference, TCC*. Springer, 2006, pp. 265–284.

[17] European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)," *Official Journal*, vol. L 110, pp. 1–88, 2016.

[18] A. Fan, M. Lewis, and Y. Dauphin, "Hierarchical neural story generation," in *56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 2018, pp. 889–898.

[19] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[20] V. Feldman, "Does learning require memorization? a short tale about a long tail," in *52nd Annual ACM SIGACT Symposium on Theory of Computing*. ACM, 2020, pp. 954–959.

[21] P. Golle, "Revisiting the uniqueness of simple demographics in the us population," in *5th ACM Workshop on Privacy in Electronic Society*. ACM, 2006, pp. 77–80.

[22] A. Hoang, A. Bosselut, A. Celikyilmaz, and Y. Choi, "Efficient adaptation of pretrained transformers for abstractive summarization," *arXiv preprint arXiv:1906.00138 [cs.CL]*, 2019.

[23] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. v. d. Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556 [cs.LG]*, 2022.

[24] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," 2020.

[25] E. Hovy, M. Marcus, M. Palmer, L. Ramshaw, and R. Weischedel, "OntoNotes: The 90% solution," in *Hu-*

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

man Language Technology Conference of the NAACL. ACL, 2006, pp. 57–60.

[26] J. Huang, H. Shao, and K. C.-C. Chang, "Are large pre-trained language models leaking your personal information?" in *Findings of the Association for Computational Linguistics, EMNLP*. ACL, 2022, pp. 2038–2047.

[27] T. Humphries, S. Oya, L. Tulloch, M. Rafuse, I. Goldberg, U. Hengartner, and F. Kerschbaum, "Investigating membership inference attacks under data dependencies," *arXiv preprint arXiv:2010.12112 [cs.CR]*, 2020.

[28] H. A. Inan, O. Ramadan, L. Wutschitz, D. Jones, V. Rühle, J. Withers, and R. Sim, "Training data leakage analysis in language models," *arXiv preprint arXiv:2101.05405v2 [cs.CR]*, 2021.

[29] D. Ippolito, F. Tramèr, M. Nasr, C. Zhang, M. Jagielski, K. Lee, C. A. Choquette-Choo, and N. Carlini, "Preventing verbatim memorization in language models gives a false sense of privacy," *arXiv preprint arXiv:2210.17546 [cs.LG]*, 2022.

[30] A. Jagannatha, B. P. S. Rawat, and H. Yu, "Membership inference attack susceptibility of clinical language models," *arXiv preprint arXiv:2104.08305 [cs.CL]*, 2021.

[31] M. Jagielski, O. Thakkar, F. Tramèr, D. Ippolito, K. Lee, N. Carlini, E. Wallace, S. Song, A. Thakurta, N. Papernot *et al.*, "Measuring forgetting of memorized training examples," *arXiv preprint arXiv:2207.00099 [cs.LG]*, 2022.

[32] B. Jayaraman, E. Ghosh, M. Chase, S. Roy, H. Inan, W. Dai, and D. Evans, "Combing for credentials: Active pattern extraction from smart reply," *arXiv preprint arXiv:2203.07618 [cs.CR]*, 2022.

[33] N. Kandpal, E. Wallace, and C. Raffel, "Deduplicating training data mitigates privacy risks in language models," in *39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 10 697–10 707.

[34] B. Klimt and Y. Yang, "Introducing the Enron corpus," in *International Conference on Email and Anti-Spam, CEAS*, vol. 45, 2004, pp. 92–96.

[35] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, "Neural architectures for named entity recognition," in *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT*. ACL, 2016, pp. 260–270.

[36] J. Lee, T. Le, J. Chen, and D. Lee, "Do language models plagiarize?" *arXiv preprint arXiv:2203.07618 [cs.CL]*, 2022.

[37] E. Lehman, S. Jain, K. Pichotta, Y. Goldberg, and B. C. Wallace, "Does BERT pretrained on clinical notes reveal sensitive data?" *arXiv preprint arXiv:2104.07762 [cs.CL]*, 2021.

[38] X. Li, F. Tramer, P. Liang, and T. Hashimoto, "Large language models can be strong differentially private learners," in *10th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2022.

[39] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692 [cs.CL]*, 2019.

[40] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *7th International Conference on Learning Representations, ICLR*. OpenReview.net, 2019.

[41] R. T. McCoy, P. Smolensky, T. Linzen, J. Gao, and A. Celikyilmaz, "How much do language models copy from their training data? Evaluating linguistic novelty in text generation using RAVEN," *arXiv preprint arXiv:2111.09509 [cs.CL]*, 2021.

[42] B. McMahan and A. Thakurta, "Federated learning with formal differential privacy guarantees," Google, Tech. Rep., 2022. [Online]. Available: https://ai.googleblog.com/2022/02/federated-learning-with-formal.html

[43] N. Miao, H. Zhou, L. Mou, R. Yan, and L. Li, "CGMH: Constrained sentence generation by metropolis-hastings sampling," in *AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 6834–6842.

[44] F. Mireshghallah, K. Goyal, A. Uniyal, T. Berg-Kirkpatrick, and R. Shokri, "Quantifying privacy risks of masked language models using membership inference attacks," *arXiv preprint arXiv:2203.03929 [cs.LG]*, 2022.

[45] F. Mireshghallah, A. Uniyal, T. Wang, D. Evans, and T. Berg-Kirkpatrick, "Memorization in NLP fine-tuning methods," *arXiv preprint arXiv:2205.12506 [cs.CL]*, 2022.

[46] D. Nadeau and S. Sekine, "A survey of named entity recognition and classification," *Lingvisticae Investigationes*, vol. 30, no. 1, pp. 3–26, 2007.

[47] M. Nadeem, A. Bethke, and S. Reddy, "StereoSet: Measuring stereotypical bias in pretrained language models," in *59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL-IJCNLP*. ACL, 2021, pp. 5356–5371.

[48] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders *et al.*, "WebGPT: Browser-assisted question-answering with human feedback," *arXiv preprint arXiv:2112.09332 [cs.CL]*, 2021.

[49] R. Parikh, C. Dupuy, and R. Gupta, "Canary extraction in natural language understanding models," in *60th Annual Meeting of the Association for Computational Linguistics, ACL*. ACL, 2022, pp. 552–560.

[50] I. Pilán, P. Lison, L. Øvrelid, A. Papadopoulou, D. Sánchez, and M. Batet, "The text anonymization benchmark (TAB): A dedicated corpus and evaluation framework for text anonymization," *Computational Linguistics*, vol. 48, no. 4, pp. 1053–1101, 2022.

[51] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners." [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

[52] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu *et al.*, "Exploring the limits of transfer learning with a unified text-to-text transformer." *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.

[53] L. Rocher, J. M. Hendrickx, and Y.-A. De Montjoye, "Estimating the success of re-identifications in incomplete datasets using generative models," *Nature communications*, vol. 10, no. 1, pp. 1–9, 2019.

[54] P. Samarati and L. Sweeney, "Protecting privacy when disclosing information: k-anonymity and its enforcement through generalization and suppression," Computer Science Laboratory, SRI International, Tech. Rep., 1998. [Online]. Available: http://www.csl.sri.com/papers/sritr-98-04/

[55] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," *arXiv preprint arXiv:1508.07909 [cs.CL]*, 2015.

[56] W. Shi, A. Cui, E. Li, R. Jia, and Z. Yu, "Selective differential privacy for language modeling," in *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NACACL-HLT*. ACL, 2022, pp. 2848–2859.

[57] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *37th IEEE symposium on security and privacy (S&P)*. IEEE, 2017, pp. 3–18.

[58] P. Stock, I. Shilov, I. Mironov, and A. Sablayrolles, "Defending against reconstruction attacks with Rényi differential privacy," *arXiv preprint arXiv:2202.07623 [cs.LG]*, 2022.

[59] K. Tirumala, A. H. Markosyan, L. Zettlemoyer, and A. Aghajanyan, "Memorization without overfitting: Analyzing the training dynamics of large language models," in *Advances in neural information processing systems (NeurIPS)*, vol. 35. Curran Associates, Inc., 2022, pp. 38 274–38 290.

[60] T. Vakili and H. Dalianis, "Are clinical BERT models privacy preserving? The difficulty of extracting patient-condition associations." in *AAAI 2021 Fall Symposium on Human Partnership with Medical AI: Design, Operationalization, and Ethics (AAAI-HUMAN)*. CEUR-WS.org, 2021.

[61] T. Vakili, A. Lamproudis, A. Henriksson, and H. Dalianis, "Downstream task performance of BERT models pre-trained using automatically de-identified clinical data," in *13th Language Resources and Evaluation Conference, LREC*. ACL, 2022, pp. 4245–4252.

[62] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems (NeurIPS)*, vol. 30, p. 6000–6010, 2017.

[63] L. Wutschitz, H. A. Inan, and A. Manoel, "dp-transformers: Training transformer models with differential privacy." [Online]. Available: https://www.microsoft.com/en-us/research/project/dp-transformers

[64] S. Yeom, I. Giacomelli, A. Menaged, M. Fredrikson, and S. Jha, "Overfitting, robustness, and malicious algorithms: A study of potential causes of privacy risk in machine learning," *Journal of Computer Security*, vol. 28, no. 1, pp. 35–70, 2020.

[65] A. Yousefpour, I. Shilov, A. Sablayrolles, D. Testuggine, K. Prasad, M. Malek, J. Nguyen, S. Ghosh, A. Bharadwaj, J. Zhao *et al.*, "Opacus: User-friendly differential privacy library in PyTorch," *arXiv preprint arXiv:2109.12298 [cs.LG]*, 2021.

[66] D. Yu, S. Naik, A. Backurs, S. Gopi, H. A. Inan, G. Kamath, J. Kulkarni, Y. T. Lee, A. Manoel, L. Wutschitz, S. Yekhanin, and H. Zhang, "Differentially private fine-tuning of language models," in *10th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2022.

[67] S. Zanella-Béguelin, L. Wutschitz, S. Tople, V. Rühle, A. Paverd, O. Ohrimenko, B. Köpf, and M. Brockschmidt, "Analyzing information leakage of updates to natural language models," in *27th ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 363–375.

[68] R. Zhang, S. Hidano, and F. Koushanfar, "Text revealer: Private text reconstruction via model inversion attacks against transformers," *arXiv preprint arXiv:2209.10505 [cs.CL]*, 2022.

[69] X. Zhao, L. Li, and Y.-X. Wang, "Provably confidential language modelling," in *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NACACL-HLT*. ACL, 2022, pp. 943–955.

Appendix A
Datasets

We survey three datasets that we split into *training*, *validation* and *testing* sets. The training and validation sets are equally large and are used to train the target and shadow models respectively. We evaluate the perplexity of all models on the testing set, which neither the target nor the shadow models have seen during training.

ECHR. ECHR contains cases from the European Court of Human Rights, which consists of $118\,161$ records. Each record consists of 88.12 tokens on average. A case contains a numbered list of *facts*, which are descriptions of the case, such as persons involved or an order of events. We split the dataset so that each record contains a single fact.

Flair NER tags $16\,133$ unique PII of the entity class 'person' and we identify that 23.75% of the records contain at least one PII sequence. PII is duplicated according to a power law distribution with a mean duplication rate of 4.66. A majority of PII ($\geq 90\%$) sequences are duplicated less than 3 times. For ECHR, many PII from the 'person' class are full names (containing a first and a second name), but there are also name abbreviations (e.g., 'J.D.' instead of 'John Doe').

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

Enron. Enron contains about 600 000 real e-mails from 158 employees of the Enron Corporation. The e-mails were made public by the Federal Energy Regulatory Commission after an investigation. We observe that e-mails use informal language and may contain typographical errors. E-mails do not contain a header (e.g., a 'from', 'to' or 'title' field) unless an e-mail has been forwarded, in which case the forwarded e-mails and their headers are contained in the e-mail's body. The e-mails are typically structured with a greeting, followed by the e-mail's content, followed by a footer containing the name and personal information about the sender, such as their e-mail or work address, phone number, and homepage. We randomly sample a subset of 123 317 records from the Enron dataset, where each record consists of 346.10 tokens on average. Flair tags 105 880 unique PII sequences from the 'person' entity class, and a majority of records contain at least one PII sequence (81.45%). PII sequences have a duplication rate of 11.68 and contain an average of 3.00 tokens. From qualitative analysis, we observe that PII in the Enron dataset consists of either just the first name or a first and a second name.

Yelp-Health. The Yelp-Health dataset consists of comments made on the `yelp.com` website about facilities from the 'Health & Medical' category. Facilities from this category include Psychoanalysts, Dental Hygienists, and Cardiologists among many others. Compared to ECHR and Enron, Yelp-Health reviews contain relatively short and few PII from the 'person' entity class.

We randomly sample a subset of 78 794 reviews with an average length of 143.92 tokens per record. Flair tags 17 035 pieces of PII with an average duplication rate of 5.53 and 2.17 tokens per piece. In total, 54.55% of records contain at least one PII sequence. From a qualitative analysis, we find that pieces of PII from the 'person' entity class typically only contain the first name unless they refer to the name of the doctor, who is usually addressed by their last name.

Even though reviews on Yelp-Health were knowingly posted to a public forum[7], these posts may contain sensitive information. For example, we find detailed descriptions of a relative's disease with timestamps and locations which could be used to re-identify that individual. We believe that studying leakage using datasets containing unprocessed data from users who may be unaware of the consequences resulting from re-identification is of particular interest to the community.

A. PII Definition

This subsection describes what we mean by "direct" and "quasi-identifying" PII. Our definition is equivalent to Pilán et al. [50], who study leakage of PII in large text datasets.

- *Direct identifiers*: A direct identifier is a type of PII that can be used to uniquely identify an individual within a dataset. Examples of direct identifiers include an individual's full name, social security number, address of residence, cellphone number or email address. As a result of their sensitive nature, direct identifiers are often subject to legal and regulatory frameworks aimed at safeguarding the privacy of individuals [17].
- *Quasi-identifiers*: A of PII is a quasi-identifier when it can indirectly identify an individual through its combination with other quasi-identifying information. Examples of quasi-identifiers include an individual's age, gender, ethnicity, religion, and occupation. Because of their potential to indirectly identify individuals, both direct and quasi-identifiers are considered personal information and are subject to the same legal and regulatory frameworks in order to protect the privacy of individuals [17].

B. PII Entity Classes

We group PII by the following *entity classes*[8].

1) cardinal: A cardinal value (e.g., "12").
2) ordinal: An ordinal value (e.g., "11th").
3) date: A date (e.g., "May 23rd 2015")
4) event: An event name (e.g., "Blackhat").
5) fac: A building name (e.g., "Newark Airport").
6) gpe: Geo-political entity (e.g., "UAE").
7) language: A language name (e.g., "Catalan")
8) law: Law names (e.g., "Bill C-99").
9) money: Currency (e.g., "1.25m USD")
10) norp: Affiliation (e.g., "Alaskan")
11) person: Names (e.g., "John Henry Doe")
12) loc: Location (e.g., "11th District")
13) org: Organization (e.g., "Microsoft")
14) percent: (e.g., "(+0.78%)")
15) product: (e.g., "GX-532")
16) quantity: (e.g. "2000 lbs")
17) time: (e.g., "5:30 pm EDT")
18) work of art: (e.g., "Star Wars")
19) phone number: (e.g., "(+1)123-456-7890")
20) email address: (e.g., "john.doe@anon.com")
21) url: (e.g., "www.johndoe.com")

1) Replacing PII: The attack described in Algorithm 4 involves substituting existing PII with PII of the same class. For example, in the sentence "Teo Peric is an engineer." we could replace "Teo Peric" with a different PII of the class 'person', resulting in "Ana Jaksic is an engineer.".

C. PII Distribution

Table VII show the PII counts in each of the three surveyed datasets.

D. PII Leakage Metrics

We define formally the metrics introduces in Section IV-D.

1) **PII Extractability:** We assess PII extractability using precision and recall. Precision (also called positive predictive value) quantifies the ratio of true positive predictions to all positive predictions made by a classifier, while recall is the true positive rate, i.e. the ratio of true positive predictions to all positive instances in the dataset. High recall implies that a significant proportion of PII can

[7]https://www.yelp.com/developers/documentation/v3/all_category_list

[8]https://huggingface.co/flair/ner-english-ontonotes-large

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.

TABLE VII: A summary of the evaluated datasets and statistics for PII from the entity class 'person'. This table summarizes the part of the dataset that was used to train the model, after splitting the entire dataset into equally large *training* and *validation* sets, and a small *testing* set.

	Records	Tokens / Record	Unique PII	Records w. PII	Duplicates / PII	Tokens / PII
ECHR	118 161	88.12	16 133	23.75%	4.66	4.00
Enron	138 919	346.10	105 880	81.45%	11.68	3.00
Yelp-Health	78 794	143.92	17 035	54.55%	5.35	2.17

Algorithm 9 Fill residual masks

1: **procedure** FILL-MASKS(S)
2: $i \leftarrow 0$
3: **while** `[MASK]` $\in S$ **do**
4: $S_i, S \leftarrow$ SPLIT(S, `[MASK]`) ▷ *At first* `[MASK]`
5: $i \leftarrow i + 1$
6: $S_i \leftarrow S$
7: $S' \leftarrow S_0$
8: **for** $j \leftarrow 1$ **to** i **do**
9: $w_j \leftarrow \text{MLM}(S', S_j)$
10: $S' \leftarrow S' w_j S_j$
11: **return** S'

be extracted from the LM's generated output, and high precision suggests that a generated PII is highly probable to appear in the training dataset. Algorithm 4 represents the PII extraction game, and Equation (2) defines the adversary's success as its recall. Given the number of unique PII sequences $|\mathcal{C}|$ in D, the adversary produces a set of PII sequences $\tilde{\mathcal{C}}$ with a maximum size of $|\mathcal{C}|$. PRECISION denotes the expected fraction of correctly identified PII sequences in $\tilde{\mathcal{C}}$ relative to the total number of PII sequences produced by the adversary ($|\tilde{\mathcal{C}}|$). Recall and precision are formally calculated as follows:

$$\text{RECALL} = \mathbb{E}\left[\frac{|\mathcal{C} \cap \tilde{\mathcal{C}}|}{|\mathcal{C}|}\right] \quad \text{PRECISION} = \mathbb{E}\left[\frac{|\mathcal{C} \cap \tilde{\mathcal{C}}|}{|\tilde{\mathcal{C}}|}\right] \tag{6}$$

2) **PII Reconstruction and Inference:** We measure the top-1 accuracy for PII reconstruction and inference as formalized in Algorithm 5. Given a randomly sampled sequence S from the training dataset that contains at least one piece of PII C, we compute the accuracy of an attacker to produce a guess $\tilde{C} = C$ for a randomly selected `[MASK]` within the sequence.

$$\text{Succ}_{\text{RECON}} = \Pr\left[\tilde{C} = C\right] \tag{7}$$

The success of the attack depends mildly on the presence of other PII in the sampled sequence S. This is because other PII sequences would be scrubbed and the adversary would have less contextual information to make an inference. Figure 4 illustrates this point for a sequence containing multiple pieces of PII.

APPENDIX B
FILLING RESIDUAL MASKS

Algorithm 9 describes our procedure for filling residual masks in a masked query. We use a publicly available masked language model (MLM) to fill in one masked token at a time. Given a sentence with multiple masked PII, for example "`[MASK]` plays soccer in `[MASK]`, England." the goal of FILL-MASKS is to fill in all masked tokens in a way that preserves the meaning of the sentence.

Publicly available MLMs do not jointly fill multiple tokens. (See for instance, https://huggingface.co/docs/transformers/main_classes/pipelines#transformers.FillMaskPipeline.) Hence we resort to filling in masked tokens from left to right using the top token suggested by the MLM. In the example above, we would query the MLM twice. The first query is "`[MASK]` plays soccer in" and the second is "John plays soccer in `[MASK]`, England.", assuming the MLM fills the first mask with "John".

Given a sentence $S = S_0$`[MASK]`$S_1 \ldots$`[MASK]`S_i, Algorithm 9 first identifies the subsequences $S_0, \ldots, S_i$ (lines 2–6) and then queries the MLM to fill in each masked token separating them from left to right (lines 7–10). The function $\text{MLM}(S_0, S_1)$ queries the MLM on S_0`[MASK]`S_1 to predict the most likely token in the position of `[MASK]`.

Authorized licensed use limited to: Rutgers University. Downloaded on February 19,2026 at 18:23:24 UTC from IEEE Xplore. Restrictions apply.